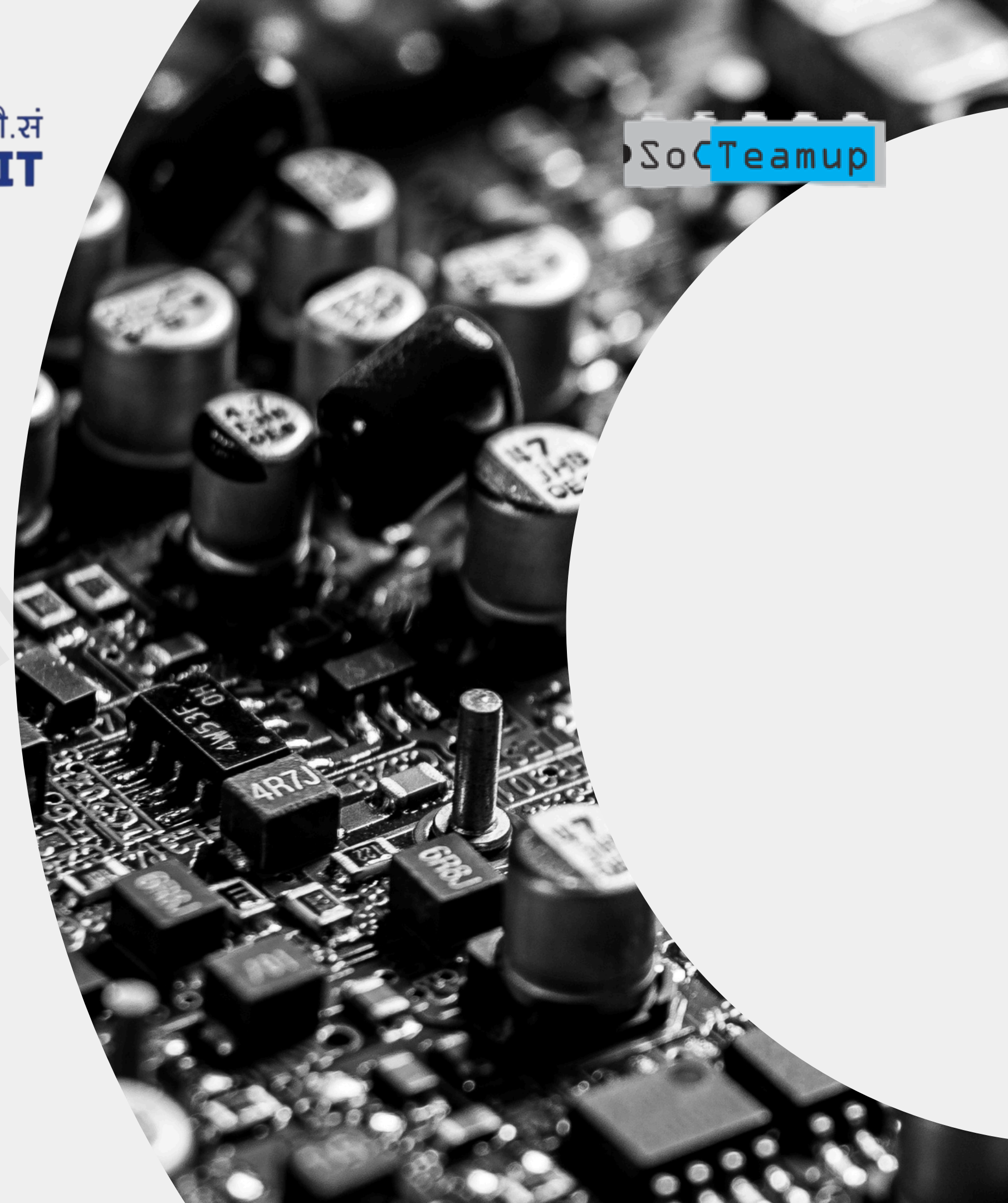


Introduction to Design and Verification tool **Verilator**





Outline

Section Title
What is Verilator?
What is GTKWave?
Key Features and Use Cases
Writing Verilog Code for Verilator
Creating a Testbench
Simulating and Viewing Waveforms
Hands-on Example: AND Gate
Tips and Best Practices
Summary

What is Verilator?

- Open-source, high-performance Verilog simulator
- Translates Verilog RTL → C++/SystemC for fast simulation
- Supports only synthesizable constructs (no delays, no PLI)
- Excellent for:
 - Linting and static checks
 - Large design simulation
 - Integrating with C++ testbenches

What is GTKWave?

- Open-source waveform viewer for *.vcd files
- Graphical interface to view signal transitions over time
- Compatible with Verilator, Icarus Verilog, and more
- Features:
 - Zoom, signal search, bookmarks
 - Drag and drop signal hierarchy
 - Glitch and timing debug

➔ Key Features and Use Cases

Feature	Verilator
Open Source	✓
Linting/Static Check	✓
Fast Simulation	✓ (Compiled C++)
GUI Waveform	✗
System Verilog support	Partial
Debug RTL Logic	✓

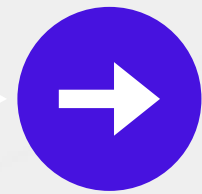
Writing Verilog for Verilator

Use:

- always @(posedge clk)
- assign statements
- module instantiation

Avoid:

- Delays like #10, \$display, \$monitor
- PLI/VPI or behavioral constructs



Example

```
module and_gate(input a, input b, output y);  
    assign y = a & b;  
endmodule
```

Creating the **Testbench**

- **Creating a Verilator-Compatible Testbench**
 - Declare testbench module
 - Instantiate DUT (Device Under Test)
 - Include **\$dumpfile** and **\$dumpvars** for waveform generation
 - Apply stimuli using initial block

Example

```
initial begin
    $dumpfile("and_wave.vcd");
    $dumpvars(0, and_gate_tb);
    a = 0; b = 0; #10;
    ...
    $finish;
end
```


Simulating and Viewing Waveforms

Terminal Commands:

```
verilator --binary -j 0 -Wall and_gate_design.v \  
and_gate_tb.v --top and_gate_tb --timing \  
--CFLAGS "-std=c++20" --trace  
gtkwave and_wave.vcd // for waveform  
analysis
```

Simulating and Viewing Waveforms

GTKWave Usage:

- Use "Add Signals" → /tb_and_gate/
- Zoom, pan, search signals
- Analyze waveforms for logic correctness

Tips and Best Practices



Use:

- Use \$dumpfile and \$dumpvars
- Limit scope of dumped signals for performance
- Use meaningful names in modules and signals
- Bookmark key waveforms in GTKWave



Avoid:

- Dumping too many signals (slows GTKWave)
- Using non-synthesizable constructs in Verilator

Thank You

echiphu

